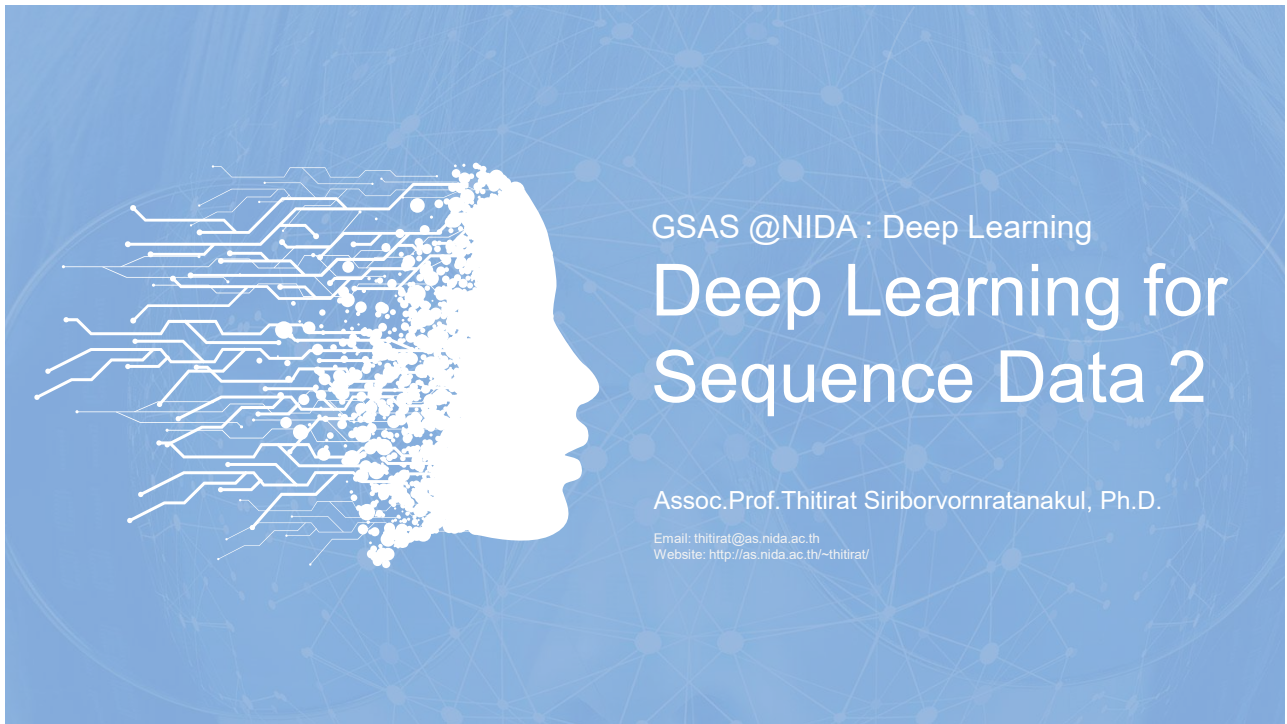




1

OUTLINE

01 Long Short-Term Memory

The practical variant of RNN for handling long(er) sequence of data.

02 Gated Recurrent Unit

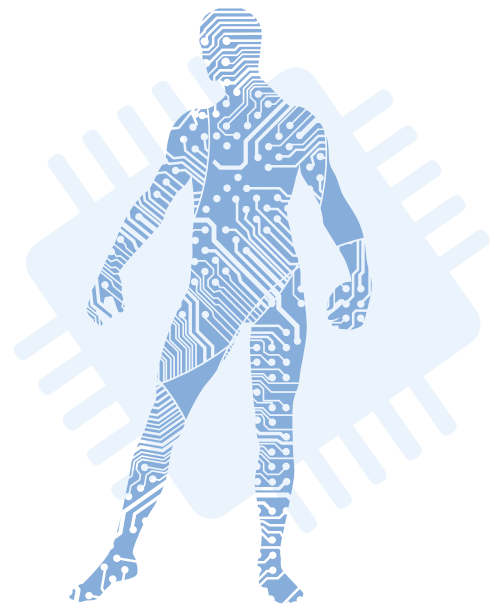
A simpler alternative of LSTM with fewer gates.

03 Other RNN techniques

Bi-directional RNN and stateful RNN.

04 Alternative architectures

Processing sequences without RNN/LSTM/GRU



2



Limitations of Vanilla RNN

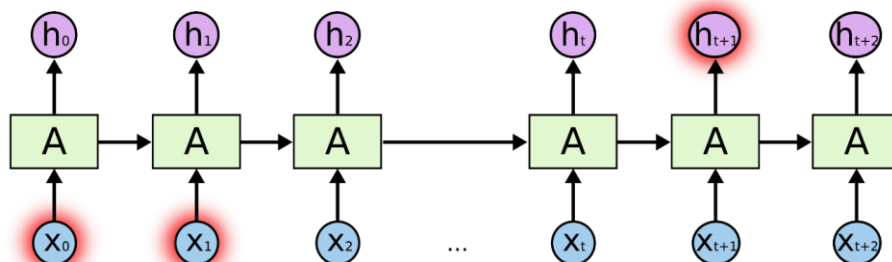
Understand why the vanilla RNN has never been used in practice.

3



Problems of Long Sequences

- To train an RNN on long sequences, we must run it over many time steps, making the unrolled RNN a very deep network.
- **In theory**, RNNs are absolutely capable of handling long sequences.

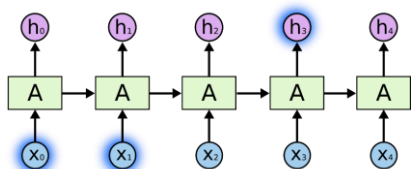


4

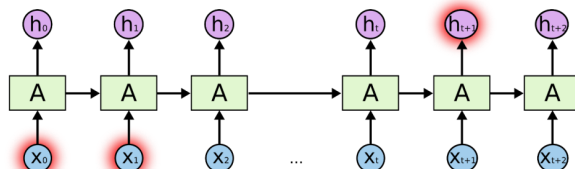
A Problems of Long Sequences

• **In practice**, there are two main problems training an RNN on long sequences:

1. RNN may take forever to train, or training may be unstable.
2. RNNs are not able to capture long-term dependencies.
 - o Meaning that when an RNN processes a long sequence, it will gradually forget the first inputs in the sequence.



RNNs can learn to use the past information when the gap between the relevant information and the place that it's needed is small.



It's entirely possible for the gap between the relevant information and the point where it is needed to become very large. Unfortunately, as that gap grows, RNNs become unable to learn to connect the information.

Image(s) from: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

5

A Problems of Long Sequences

• **Gradient Vanishing** causes RNN's states contain virtually no trace of the first inputs.

The brown and black dog, which was playing with the cat, was a german shepherd.



To predict the last two words "german" and "shepherd," the RNN would need to take into account the inputs "brown", "black", and "dog."



Break down the backpropagation error of the word "shepherd" back to "brown"

$$\frac{\partial L_{15}}{\partial \hat{y}_{15}} \frac{\partial \hat{y}_{15}}{\partial z_{15}} \frac{\partial z_{15}}{\partial h_{15}} \frac{\partial h_{15}}{\partial h_2} \frac{\partial h_2}{\partial W_x}$$

$$\frac{\partial h_{15}}{\partial h_2} = \frac{\partial h_{15}}{\partial h_{14}} \frac{\partial h_{14}}{\partial h_{13}} \dots \frac{\partial h_2}{\partial h_1}$$

That's a lot of chain rule!!!

These chains of gradients are troublesome because, if less than 1, they can cause the loss from the word "shepherd" with respect to the word "brown" to approach 0, thereby vanishing.



Credit: <https://medium.com/towards-artificial-intelligence/whirlwind-tour-of-rnns-a11effb7808f>

6

A Problems of Long Sequences

- In the context of RNNs:
 - If **gradients vanish**, a network cannot learn from the past and loses its ability to capture long-term dependencies, leading to poor generalization and underfitting.
 - If **gradients explode**, a network becomes unstable and sensitive to small changes in the input, leading to numerical overflow/underflow, erratic behavior, and overfitting.
 - Gradient exploding is more common in RNNs.

7

A Fighting the unstable gradients problem

- Many tricks we used in deep networks to deal with the unstable gradients problem, can also be used for RNNs, for example,
 - Good parameter (weight) initialization,
 - Faster optimizers,
 - Dropout,
 - ...



8

A Fighting the unstable gradients problem

- **Gradient clipping:** Clipping the gradient before propagating it backward is an easy and effective way to deal with gradient exploding.

- **Gradient clipping:**

- Simply clip the gradient values (element-wise) to a specific minimum or maximum value (e.g., $[-1, 1]$)
 - **BUT** if some elements are clipped while others aren't, the direction of the gradient may change.

- **Gradient clipping by norm:**

- Instead of element-wise clipping, lower all values in the gradient vector to be in the range (e.g., $[-1, 1]$) to maintain the value proportion and the direction of the clipped gradients.
 - **BUT** doing this may shrink some gradient values to very small values that may not be sufficient to update a deep network.

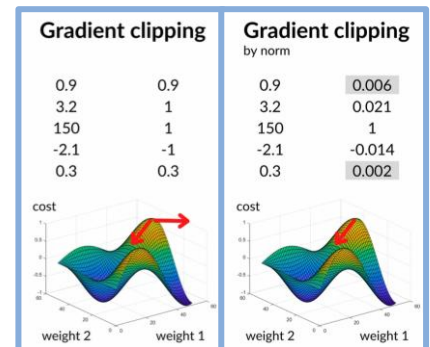
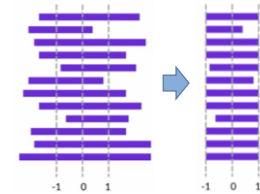


Image credit: <https://www.youtube.com/watch?v=KrQp1TxTCUY>

- Keras implementation:

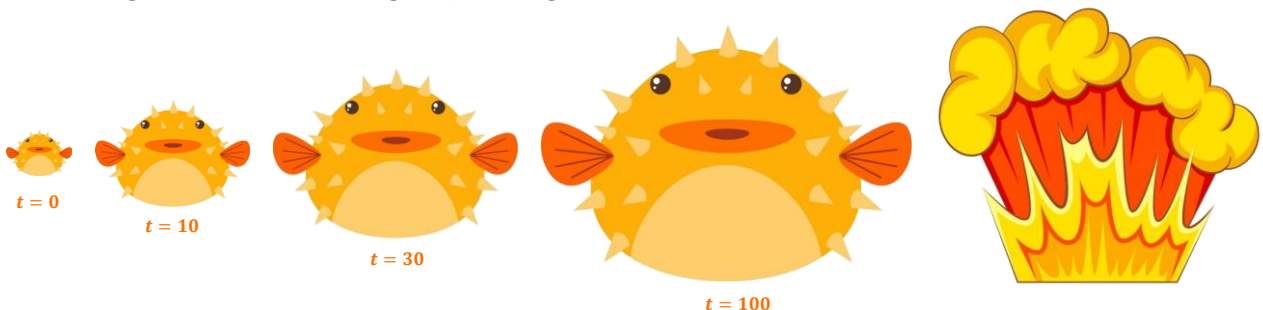
- **Gradient clipping:** Look for a `clipvalue` parameter when initializing an optimizer
 - **Gradient norm scaling:** Look for a `clipnorm` parameter when initializing an optimizer

9

A Fighting the unstable gradients problem

- **However, in case of activation function:**

- Non-saturating activation function (e.g., ReLU) may not help much in gradient vanishing/exploding of RNNs.



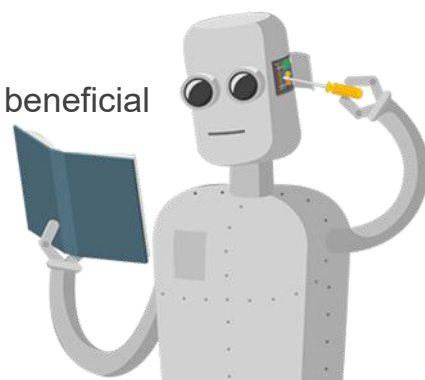
- Simply use a saturating activation function like the hyperbolic tangent (**tanh**) in RNNs.

10

A Fighting the unstable gradients problem

• Batch Normalization (BN) is good for gradient stabilization in CNNs but not in RNNs.

- Generally, we cannot use BN between time steps, only between recurrent layers, unless we create a custom layer of RNN where BN is internally applied at each time step.
- The paper* in 2015 found that BN was slightly beneficial only when it was applied to the inputs, not to the hidden states.
 - Meaning that, it is better than nothing to have BN added between recurrent layers but not within recurrent layers.



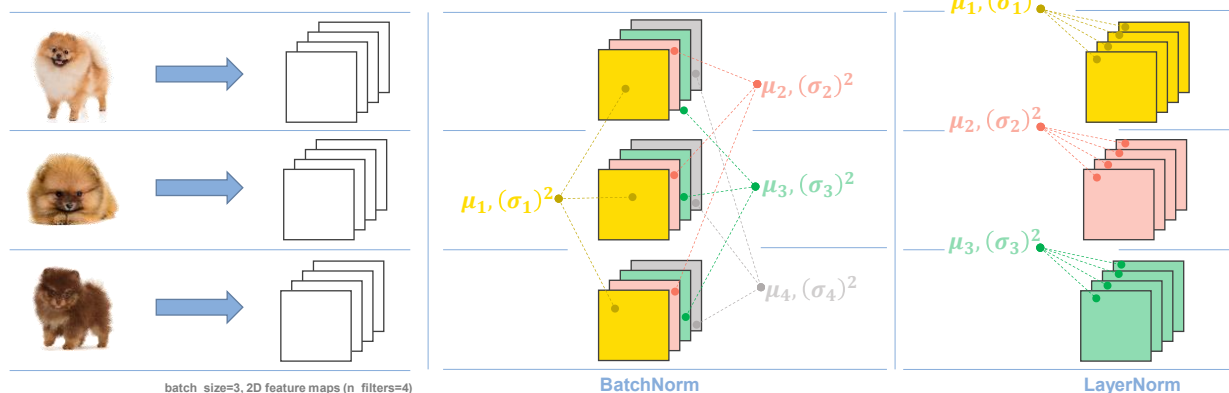
* = "Batch Normalized Recurrent Neural Networks," Proc. Of IEEE ICASSP, 2016.

11

A Fighting the unstable gradients problem

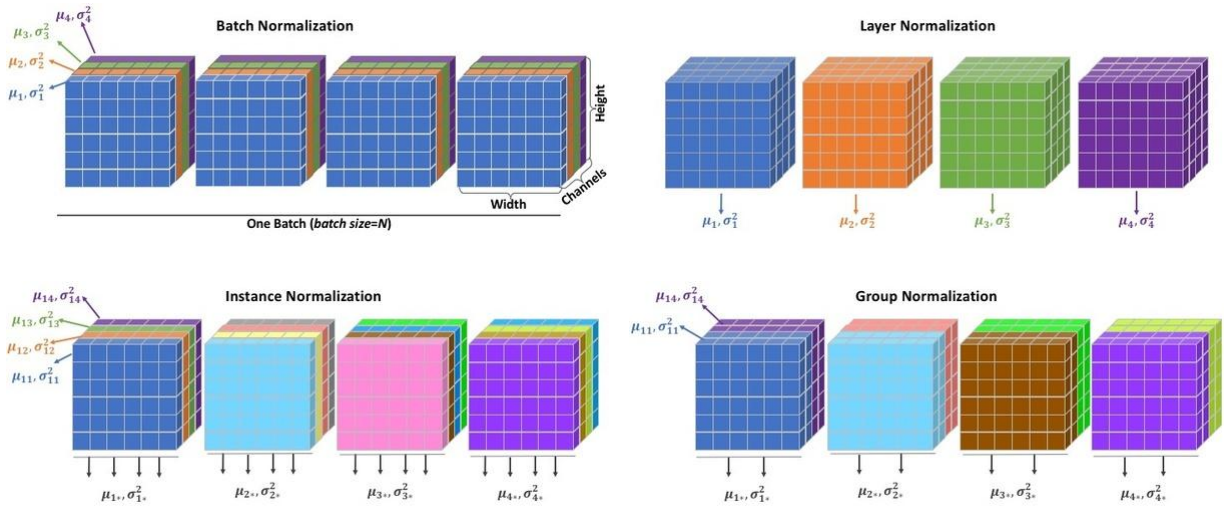
• So, how to normalize in RNNs?

- Layer Normalization (LN)(2016) often works better with RNNs as it normalizes across the features dimension.



12

A Fighting the unstable gradients problem

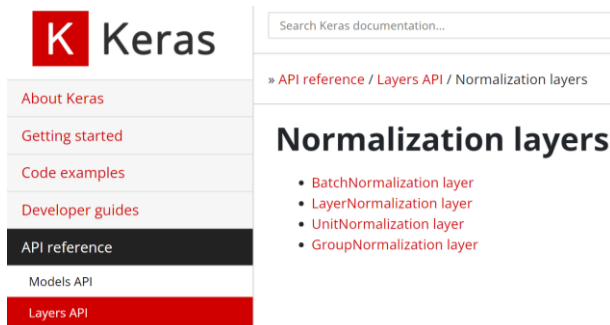


(2AUG2018) <https://www.quora.com/What-are-the-main-normalization-layers-in-artificial-neural-networks>

13

A Fighting the unstable gradients problem

• So, how to normalize in RNNs? (cont.)



```
keras.layers.LayerNormalization(
    axis=-1,
    epsilon=0.001,
    center=True,
    scale=True,
    rms_scaling=False,
    beta_initializer="zeros",
    gamma_initializer="ones",
    beta_regularizer=None,
    gamma_regularizer=None,
    beta_constraint=None,
    gamma_constraint=None,
    **kwargs
)
```

14

A Truncated BPTT (TBPTT) (1990)

- Drawbacks of conventional BPTT:
 - Saving all the inputs and the activation outputs in forward pass for use in gradient computation in the backward pass, can be computationally expensive (slow) and memory intensive, particularly with very long input sequences.
 - The accumulation of gradients over so many timesteps can result in gradient vanishing or gradient exploding problems.
- **TBPTT** is a variation of BPTT where the sequence is split into chunks. The forward pass still runs across the entire sequence, but the backward pass is truncated: gradients are computed only within each chunk and do not propagate across chunk boundaries. This yields approximate rather than fully exact gradients.
 - ✓ Reduce memory usage since only a limited number of timesteps need to be stored.
 - ✓ Lowers computational cost per update, making training faster.
 - ✓ Help mitigate exploding/vanishing gradients by restricting long chains of backpropagation.
 - ✗ Long-term dependencies may be lost since gradient does not flow across chunk boundaries.
 - ✗ Approximate gradients can lead to suboptimal convergence compared to full BPTT.
 - ✗ If ground truth is available only at the final timestep, TBPTT is especially limiting, because the gradient signal from the final loss cannot reach earlier chunks at all.

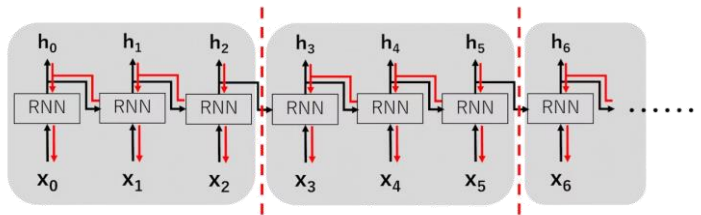


Image credit: <https://qiita.com/kibounoasa/items/78fb5a217232de863b00>

15

A Tackling the short-term memory problem

- To tackle this problem, various types of cells with long-term memory have been introduced.
- The cell with long-term memory have proven so successful that the basic RNN cells are not used much anymore.



16

AI LSTM

Long Short-Term Memory

Original LSTM paper: S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 9(8), 1997. <https://dl.acm.org/doi/10.1162/neco.1997.9.8.1735>

17

THE OLD HISTORY

- **1943:** The first **neural network** was proposed by McCulloch and Pitts.
- **1955:** John McCarthy first coined the term **Artificial Intelligence**.
- **1959:** Samuel coined and popularized the term **Machine Learning**.

- **1986:** Birth of **backpropagation**
- **1986:** Birth of **RNN** and **AE**
- **1989:** Birth of **CNN**
- **1997:** Birth of **LSTM**

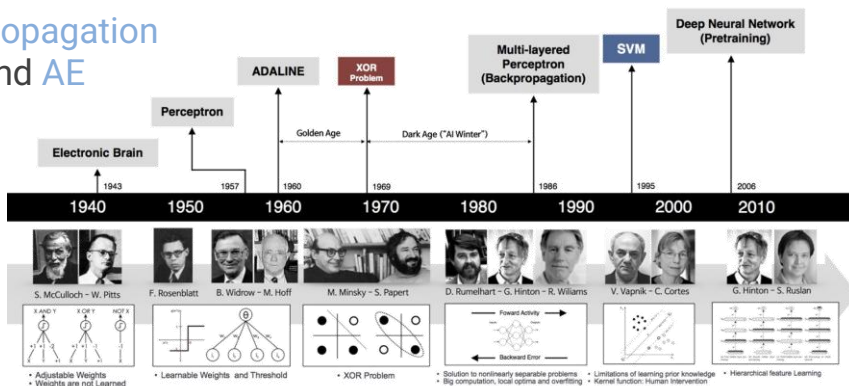


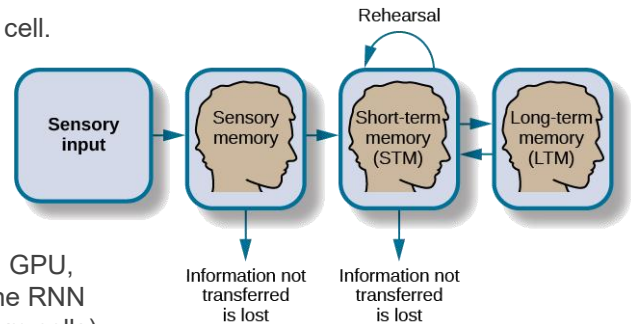
Image credit:

https://beamandrew.github.io/deeplearning/2017/02/23/deep_learning_101_part1.html

18

A LSTM (1997)

- **Long Short-Term Memory (LSTM)** is a special kind of RNN that is capable of learning long-term dependencies.
- Considering the LSTM cell as a black box:
 - It can be used very much like a basic RNN cell.
 - **BUT** LSTM will perform much better; training will converge faster, and it will detect long-term dependencies in the data.
 - **BESIDES** the LSTM layer uses an optimized implementation when running on GPU, so in general, LSTM is preferable to use (the RNN layer is mostly useful when we define custom cells).

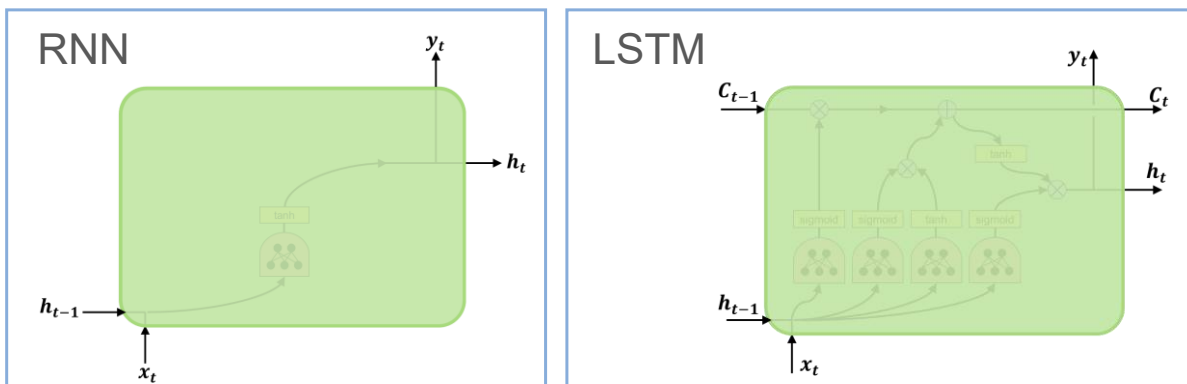


Image(s) from: <https://courses.lumenlearning.com/wmopen-psychology/chapter/reading-storage/>

19

A Vanilla RNN (1986) vs. LSTM (1997)

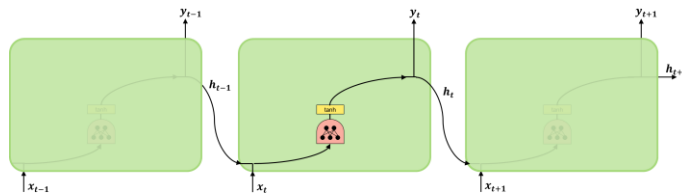
- Without looking inside, the LSTM cell looks exactly like a regular RNN cell, except that its state is split into two vectors:
 - h_t (hidden state): we can think of it as the short-term state.
 - C_t (cell state): we can think of it as the long-term state.



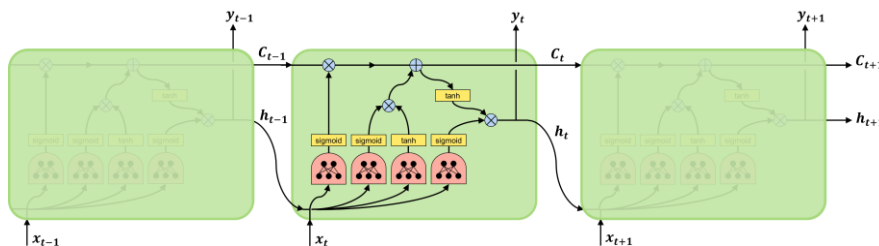
20

A Vanilla RNN (1986) vs. LSTM (1997)

- In vanilla RNNs, the repeating module will have a very simple structure, such as a single tanh layer.



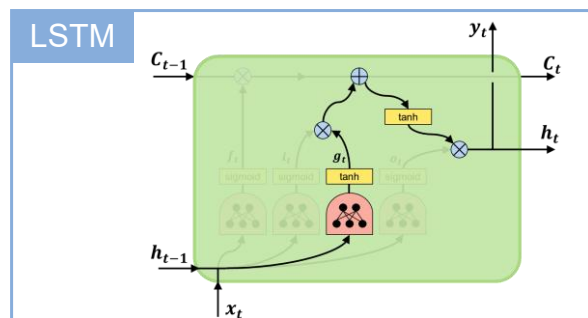
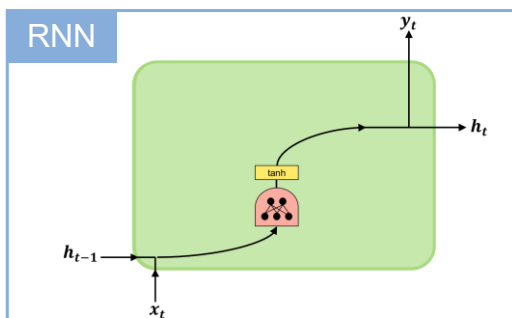
- Although LSTMs have the same chain-like structure as vanilla RNNs, the repeating module in LSTM has four neural network layers, interacting in a very special way.



21

A Vanilla RNN (1986) vs. LSTM (1997)

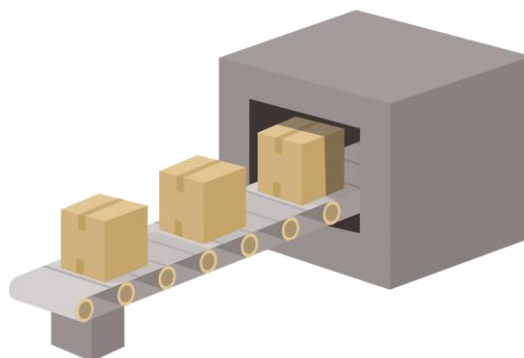
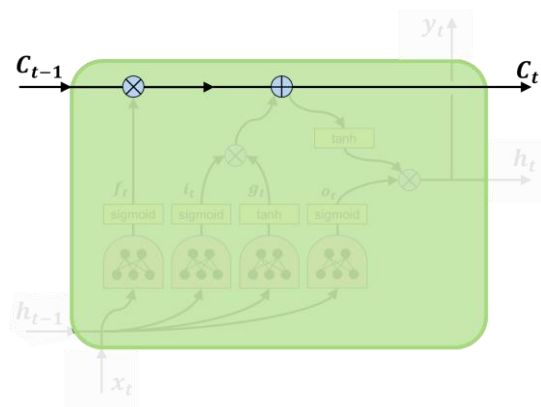
- The main layer of LSTM is the one that outputs g_t . It has the usual role of analyzing the current input x_t and the previous short-term state h_{t-1} .
 - In a basic RNN cell, there is nothing other than this layer, and its output goes straight out to y_t and h_t .
 - In contrast, in an LSTM cell this layer's output does not go straight out, but instead its most important parts are stored in the long-term state (and the rest is dropped).



22

A The core idea of LSTM

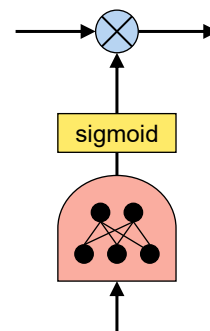
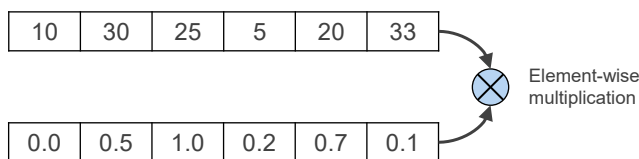
- The key to LSTMs is the **cell state**, the horizontal line running through the top of the diagram.
 - The cell state is kind of like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.



23

A The core idea of LSTM

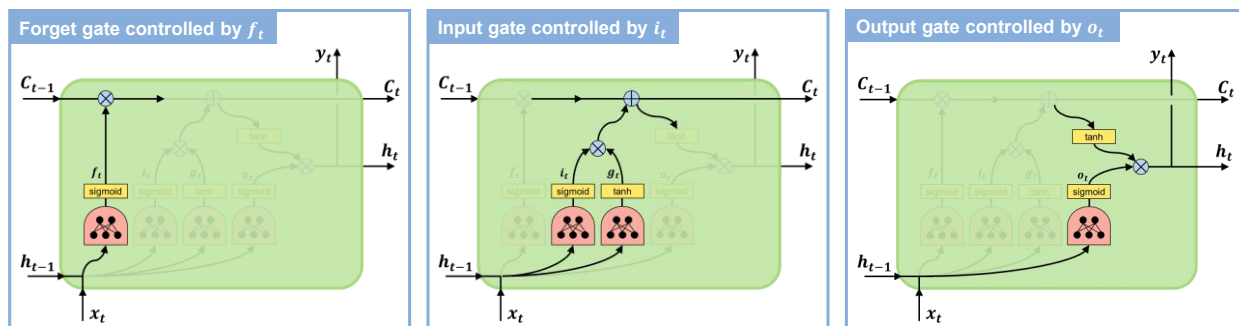
- LSTMs have the ability to remove or add information to the cell state, carefully regulated by structures called **gates**.
- **Gates** are a way to optionally let information through. They are composed out of a **sigmoid neural net layer** and an **element-wise multiplication operation**.
 - The sigmoid layer outputs numbers between zero and one, describing how much of each component should be let through. A value of zero means “let nothing through,” while a value of one means “let everything through!”



24

The core idea of LSTM

- In LSTM, there are three gate controllers to learn what to store in the long-term state, what to throw away, and what to read from it.
 - The **forget gate** (f_t) controls which parts of the long-term state should be erased.
 - The **input gate** (i_t) controls which parts of g_t should be added to the long-term state.
 - The **output gate** (o_t) controls which parts of the long-term state should be read and output at this time step, both to h_t and to y_t .



25

Reminder: RNN's equation

EX. Train RNN to predict the letter “s” given the letters “d”-“o”-“g”.

- $k = 4$: the input x is a 4-dimensional one-hot vector for the letters in “dogs”.
- $d = 3$: here we use 3 hidden nodes for simplicity.

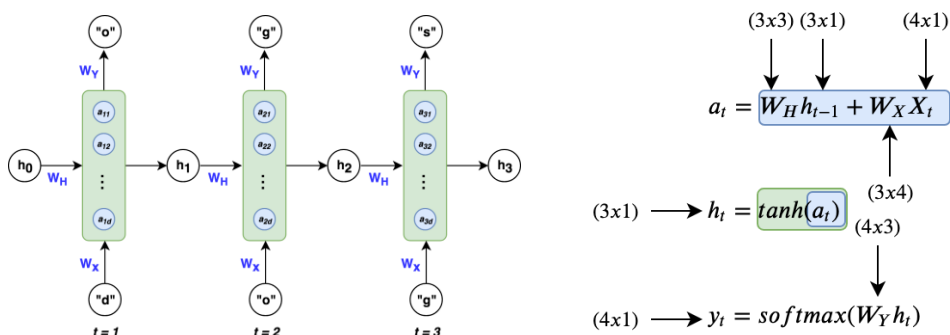
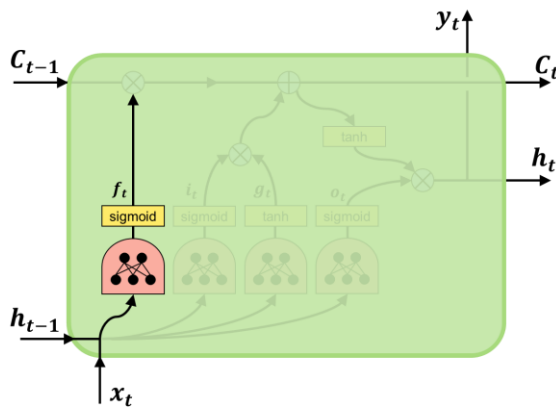


Image credit: <https://medium.com/towards-artificial-intelligence/whirlwind-tour-of-rnns-a11effb7808f>

26

LSTM Walk-Through

- The first step in LSTM is to decide **what information to throw away from the cell state**.
 - This decision is made by the forget gate (f_t).



$$f_t = \sigma(W_{f,x} x_t + W_{f,h} h_{t-1} + b_f)$$

Trainable parameters

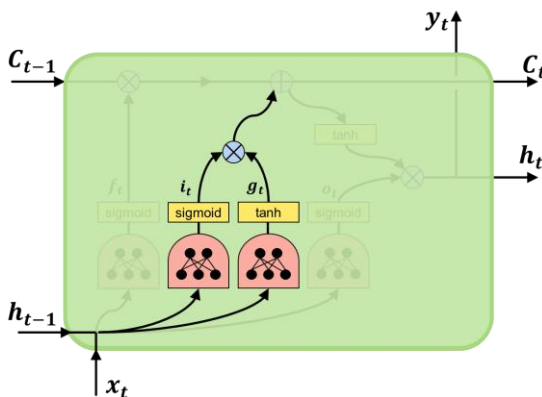
Ex A language model trying to predict the next word based on all the previous ones.

- In this problem, the cell state might include the gender of the present subject, so that the correct pronouns can be used.
- When we see a new subject, we want to forget the gender of the old subject.

27

LSTM Walk-Through

- The next step is to decide **what new information to store in the cell state**.
 - A tanh layer creates a vector of new candidate values, g_t , that could be added to the cell state.
 - The input gate (i_t) decides which parts of g_t should be added to the cell state.



$$g_t = \tanh(W_{g,x} x_t + W_{g,h} h_{t-1} + b_g)$$

$$i_t = \sigma(W_{i,x} x_t + W_{i,h} h_{t-1} + b_i)$$

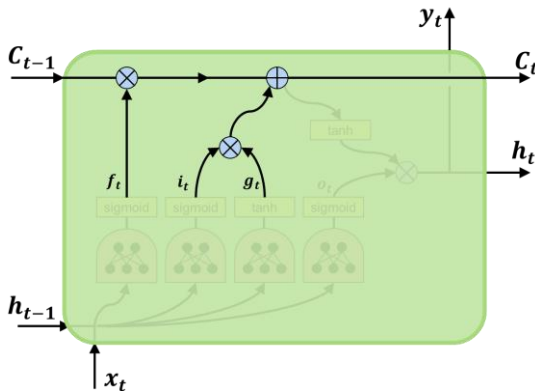
Ex A language model trying to predict the next word based on all the previous ones.

- When we want to add the gender of the new subject to the cell state, in order to replace the old one we're forgetting.

28

LSTM Walk-Through

- 3 Update the old cell state C_{t-1} to the new cell state C_t according to 1 and 2.
1. Multiply the old state C_{t-1} by f_t to forget the things we decided to forget earlier.
 2. Then add $i_t * g_t$ to the result of 1 in order to get the new candidate values, scaled by how much we decided to update each state value.



$$C_t = (f_t * C_{t-1}) + (i_t * g_t)$$

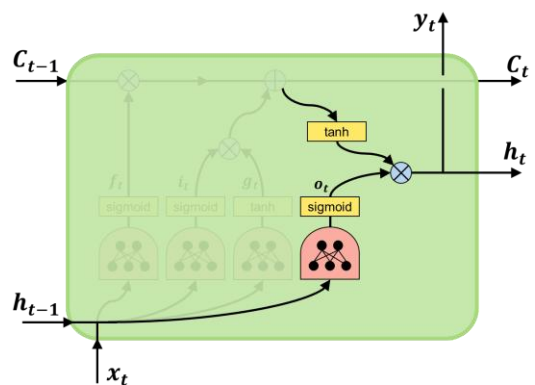
Ex A language model trying to predict the next word based on all the previous ones.

- This is when we actually drop the information about the old subject's gender and add the new information, as decided in the previous steps.

29

LSTM Walk-Through

- 4 Decide what to output (make new prediction).
1. Put the updated cell state C_t through $\tanh$ (to push the values to between -1 and 1).
 2. The output gate (o_t) decides which parts of C_t we're going to output.
 3. Multiply the results from 1 and 2, so that we only output the parts we decided to.



$$o_t = \sigma(W_{o,x} x_t + W_{o,h} h_{t-1} + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Ex A language model trying to predict the next word based on all the previous ones.

- Since it just saw a subject, it might want to output information relevant to a verb, in case that is what's coming next.
- For example, it might output whether the subject is singular or plural, so that we know what form a verb should be conjugated into if that's what follows next.

30

François Chollet

(Keras creator,
TensorFlow contributor
and Google AI
researcher)

"In summary, you don't need to understand everything about the specific architecture of an LSTM cell; as a human, it shouldn't be your job to understand it.

Just keep in mind what the LSTM cell is meant to do: allow past information to be reinjected at a later time."



31



GRU

Gated Recurrent Unit

Original GRU paper: Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," EMNLP 2014 <https://arxiv.org/abs/1406.1078>

32

AI THE MODERN HISTORY

- **2006:** The **pretraining** technique by Hinton et al.



- **2012:** ImageNet evolution—**AlexNet** 

- **2014:** Birth of **GRU**, **VAE**, and **GAN**

- **2015:** Birth of **diffusion model**

- **2017:** The **DeepFake** viral

- **2017:** Birth of the groundbreaking **Transformer**



- **2018:** NLP's ImageNet moment—**BERT**

- **2018:** ACM Turing Award to Deep NN

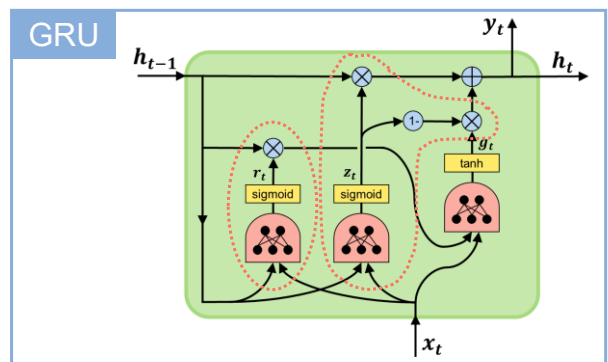
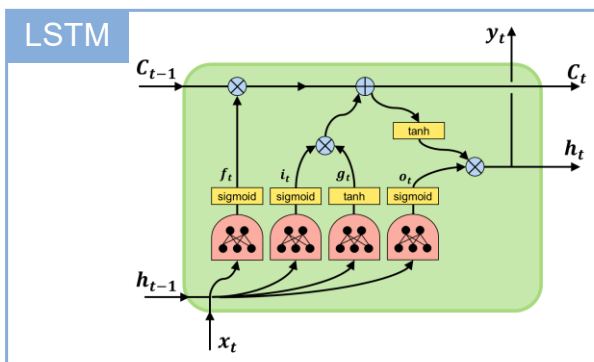


33

AI LSTM (1997) vs. GRU (2014)

- **Gated Recurrent Unit (GRU)** is the newer generation of RNNs and is very similar to LSTM.

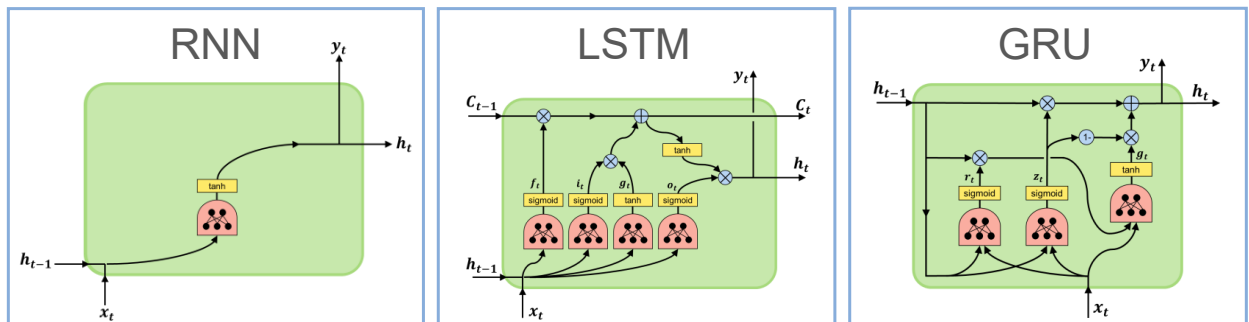
- GRU's got rid of the cell state and used the hidden state to transfer information.
- GRU only has two gates, a **reset gate** (decide how much past information to forget) and an **update gate** (decide what information to throw away and what new information to add).



34

A LSTM (1997) vs. GRU (2014)

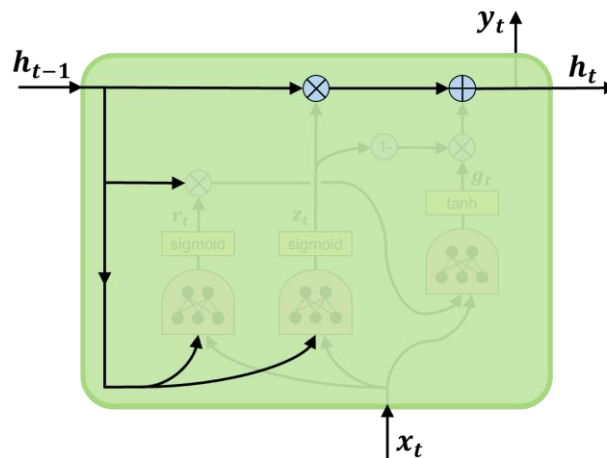
- **GRU**'s has fewer tensor operations; therefore, they are a little speedier to train than LSTM's. **GRU** seems to perform just as well as LSTM (which explains its growing popularity).
- There isn't a clear winner whether LSTM or **GRU** is better. Researchers and engineers usually try both to determine which one works better for their use case.



35

A The core idea of GRU

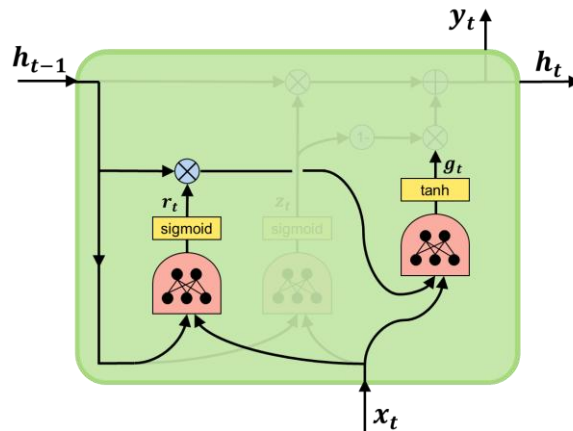
- The GRU cell is a simplified version of the LSTM cell.
- Both state vectors of LSTM are merged into a single vector h_t in GRU.



36

The core idea of GRU

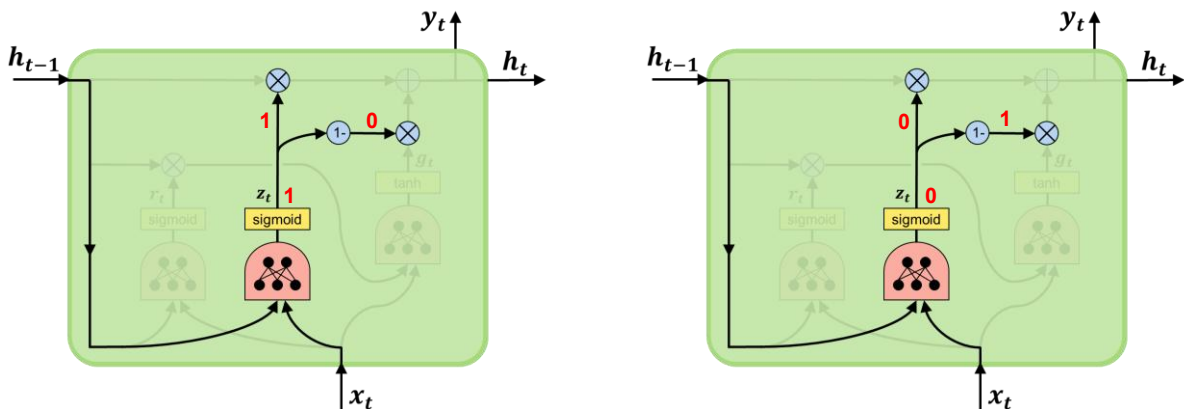
- There is no output gate; the full state vector is output at every time step. However, there is a new gate controller z_t that controls which part of the previous state (h_{t-1}) will be shown to the main layer (g_t).



37

The core idea of GRU

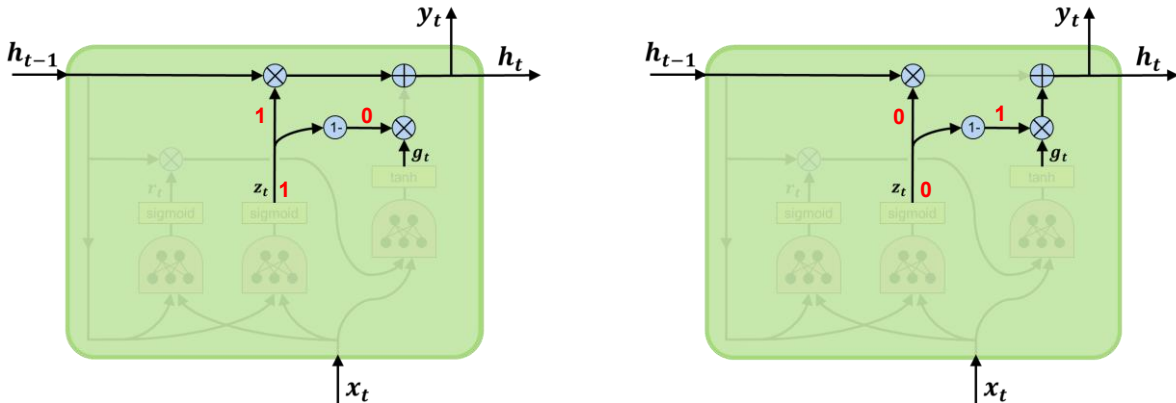
- A single gate controller z_t controls both the forget gate and the input gate.
 - If the gate controller outputs a 1, the forget gate is open ($= 1$) and the input gate is closed ($1-1 = 0$). If the gate controller outputs a 0, the opposite happens.



38

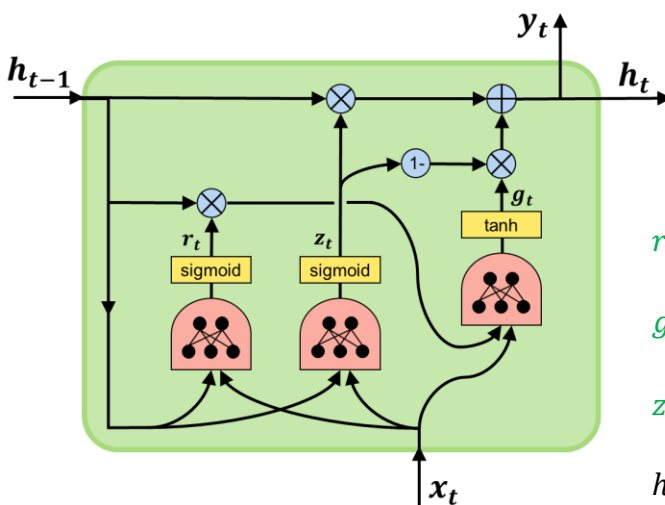
The core idea of GRU

- A single gate controller z_t controls both the forget gate and the input gate.
 - In other words, whenever a memory must be stored, the location where it will be stored is erased first.



39

The core idea of GRU



$$r_t = \sigma(W_{r,x} x_t + W_{r,h} h_{t-1} + b_r)$$

$$g_t = \tanh(W_{g,x} x_t + W_{g,h} (h_{t-1} * r_t) + b_g)$$

$$z_t = \sigma(W_{z,x} x_t + W_{z,h} h_{t-1} + b_z)$$

$$h_t = (z_t * h_{t-1}) + [(1 - z_t) * g_t]$$

40



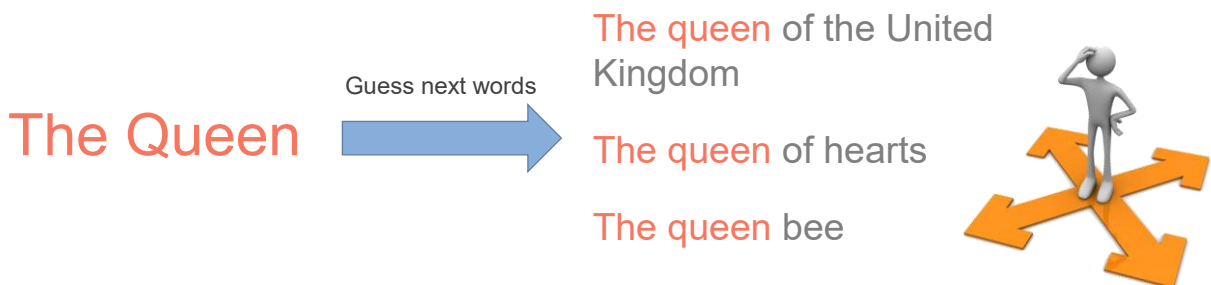
Other RNN techniques

Bi-directional RNN and stateful RNN.

41

Uni-Directional RNN

- Limitations of regular recurrent layer (uni-directional RNN):
 - It only looks at past and present inputs before generating its output. In other words, it cannot look into the future.
 - This type of RNN makes sense when forecasting time series, but for many NLP tasks, such as machine translation, it is often preferable to look ahead at the next words before encoding a given word.

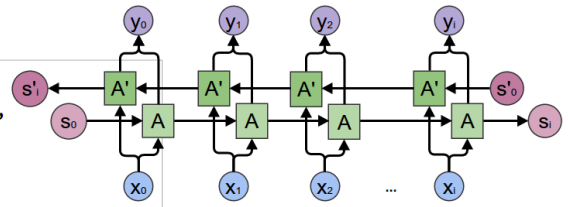


42

Bi-Directional RNN (1997)

- **Bidirectional Recurrent Neural Networks (BRNN)** are just putting two independent RNNs/LSTMs/GRUs together.
 - The input sequence is fed in normal time order (e.g., input from left to right) for one network, and in reverse time order (e.g., input from right to left) for another.
 - The outputs of the two networks are usually concatenated at each time step, though there are other options (merge_mode = {'sum', 'mul', '**concat**', 'ave', None}).
 - In keras, just wrap a recurrent layer in a `keras.layers.Bidirectional` layer.

```
model = Sequential()
model.add(Bidirectional(LSTM(10, return_sequences=True),
                       input_shape=(5, 10)))
model.add(Bidirectional(LSTM(10)))
model.add(Dense(5))
model.add(Activation('softmax'))
model.compile(loss='categorical_crossentropy', optimizer='rmsprop')
```



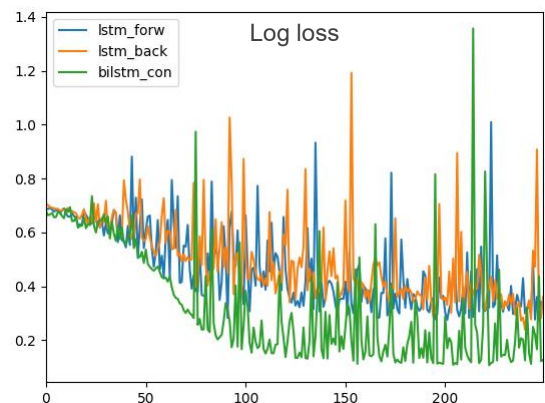
Image(s) from: <https://towardsdatascience.com/understanding-bidirectional-rnn-in-pytorch-5bd25a5dd66>

43

Bi-Directional RNN (1997)

- Interesting article:
 - (8JAN2020) How to Develop a Bidirectional LSTM For Sequence Classification in Python with Keras <https://machinelearningmastery.com/develop-bidirectional-lstm-sequence-classification-python-keras/>
 - *"This comparison will help to show that bidirectional LSTMs can in fact add something more than simply reversing the input sequence."*

By setting `go_backwards=True` in SimpleRNN / LSTM / GRU.



44

A Stateless vs. Stateful RNN

- By default, the three RNN layers are **stateless** (`stateful=False`).

```
keras.layers.SimpleRNN(
    units,
    activation="tanh",
    use_bias=True,
    kernel_initializer="glorot_uniform",
    recurrent_initializer="orthogonal",
    bias_initializer="zeros",
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    unroll=False,
    seed=None,
    **kwargs
)

keras.layers.LSTM(
    units,
    activation="tanh",
    recurrent_activation="sigmoid",
    use_bias=True,
    kernel_initializer="glorot_uniform",
    recurrent_initializer="orthogonal",
    bias_initializer="zeros",
    unit_forget_bias=True,
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    seed=None,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    unroll=False,
    use_cudnn="auto",
    **kwargs
)

keras.layers.GRU(
    units,
    activation="tanh",
    recurrent_activation="sigmoid",
    use_bias=True,
    kernel_initializer="glorot_uniform",
    recurrent_initializer="orthogonal",
    bias_initializer="zeros",
    kernel_regularizer=None,
    recurrent_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    recurrent_constraint=None,
    bias_constraint=None,
    dropout=0.0,
    recurrent_dropout=0.0,
    seed=None,
    return_sequences=False,
    return_state=False,
    go_backwards=False,
    stateful=False,
    unroll=False,
    reset_after=True,
    use_cudnn="auto",
    **kwargs
)
```

45

A Stateless vs. Stateful RNN

- stateful**: Boolean (default False). If True, the last state for each sample at index i in a batch will be used as initial state for the sample of index i in the following batch.
- In a **stateless** RNN layer, the inner state and all the outputs of one row/sequence are deleted when the sequence is processed (i.e., when one batch is processed).

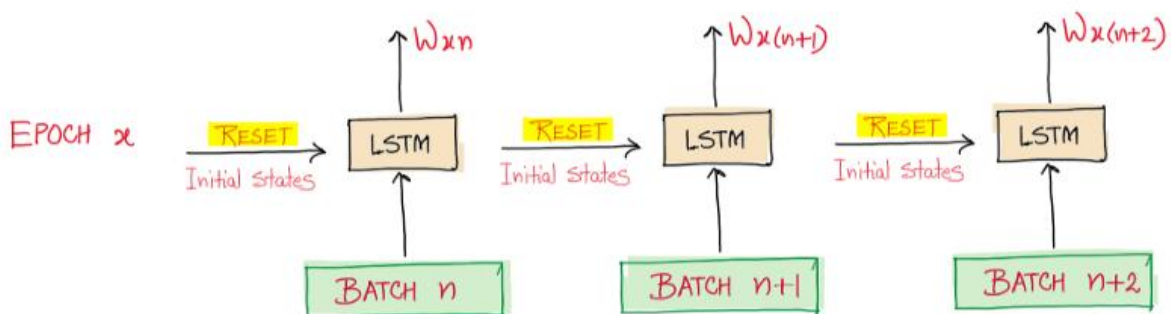


Image credit: <https://pub.towardsai.net/stateless-vs-stateful-lstms-265c83116c49>

46

A Stateless vs. Stateful RNN

- In a **stateful** RNN layer (`stateful=True`), the inner state and the outputs after each batch are deleted after each epoch, meaning that we use and update one internal state for one sequence across multiple batches.

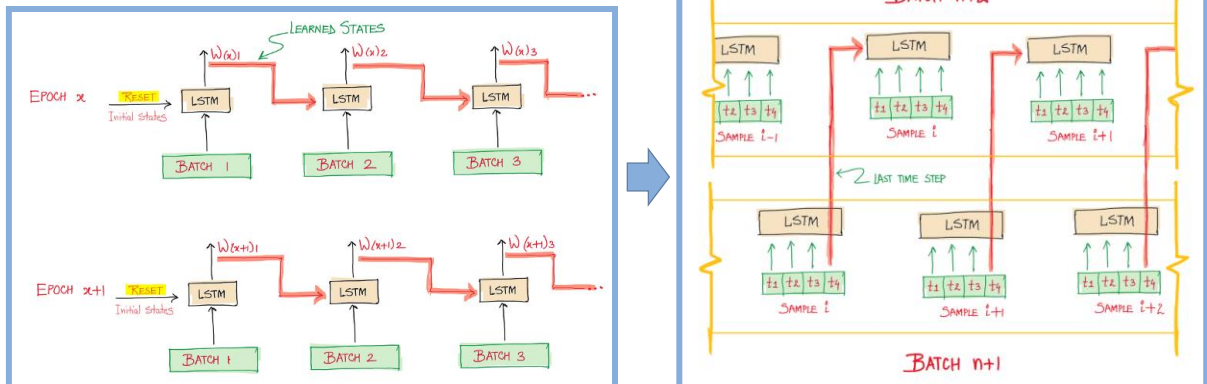


Image credit: <https://pub.towardsai.net/stateless-vs-stateful-lstms-265c83116c49>

47

A Stateless vs. Stateful RNN

- The advantage of **stateless** RNN models is that they are more performant. We need to train a **stateful** RNN model manually one epoch at a time, resetting the inner state after each epoch with `reset_states()`.
- When to use the **stateful** RNNs?
 - Computational constraints:**
 - Stateful** is used when RNNs can't process the entire sequence at once, so it's "split up" - or when different gradients are desired from backpropagation
 - Online machine learning:**
 - When it comes to real-time analysis of data in connection with incremental learning, we don't want to reset our inner state. Rather we would like to keep and update it continuously whenever a new input is collected.

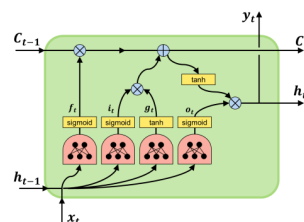
48



HANDS ON

RNN/LSTM/GRU in Keras

49



```
keras.layers.LSTM( units,
                    activation='tanh',
                    recurrent_activation='sigmoid',
                    return_sequences=False, return_state=False )
```

Activation function applied to cell states and hidden states.

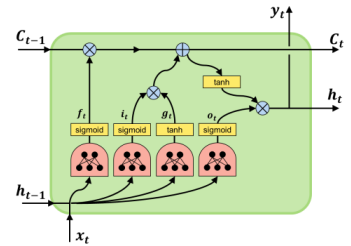
Activation function applied to the input, forget, and output gates of LSTM.



50

The dimension of the cell states in LSTM.

- In LSTM, the dimension of cell states (C_t and C_{t-1}), output mask (o_t) and hidden/output state (h_t) should have the **SAME** dimension.
- Hence, it can be said that the `units` variable represents the output's dimension of LSTM.



```
keras.layers.LSTM( units,
                  activation='tanh',
                  recurrent_activation='sigmoid',
                  return_sequences=False, return_state=False )
```

`return_sequences` refers to return the hidden state (h_t).

- By default, `return_sequences` is set to `False` in Keras, meaning that the LSTM layer will only return the last hidden state output (h_T). The last hidden state output (h_T) captures an abstract representation of the input sequence. (**Many-to-one**)
- It is possible to access the hidden state output for each input time step by setting the `return_sequences` to `True`.
- Generally, we must set `return_sequences=True` when stacking RNN/LSTM/GRU layers

- Keras provides the `return_state` argument to the SimpleRNN/LSTM/GRU layer that will provide access to the hidden state output (h_t) and the cell state (C_t).
- Generally, we do not need to access the cell state unless we are developing sophisticated models where subsequent layers may need to have their cell state initialized with the final cell state of another layer, such as in an encoder-decoder model.



51

EX1: Guess the next number (revisited 2)

64, 66, ?

- Summary:
 - Reuse most codes in `RNN_ex1` of the previous class
 - Replace `keras.layers.SimpleRNN` with either `keras.layers.LSTM` or `keras.layers.GRU`

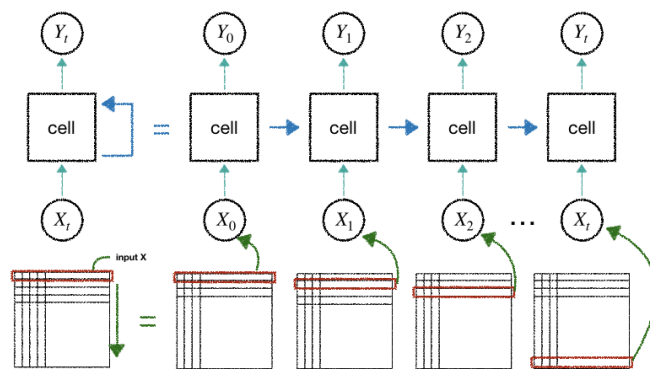
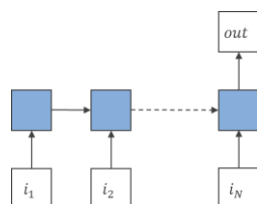


52

EX2: LSTM for MNIST Digit Classification

- Generally, CNN is said to be more suitable for the image classification task.
- This example trains LSTM with a simple MNIST dataset that we are already familiar with.
 - Treat each image as a sequence of 28 time-steps (28 rows) of 28 features (28 pixels) each.

- Use a many-to-One LSTM:

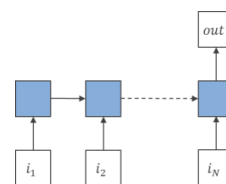


Image(s) from: <https://woolulu.tistory.com/129>

53

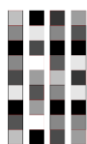
EX3: IMDB Movie Review Classification

- Create a many-to-one LSTM model
- Use `keras.layers.Embedding()` to embed one positive integer index (of one word) into a dense vector (64D) of floating points.



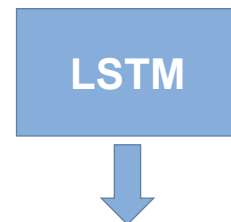
One-hot word vectors:

- Sparse
- High-dimensional
- Hard-coded



Word embeddings:

- Dense
- Lower-dimensional
- Learned from data

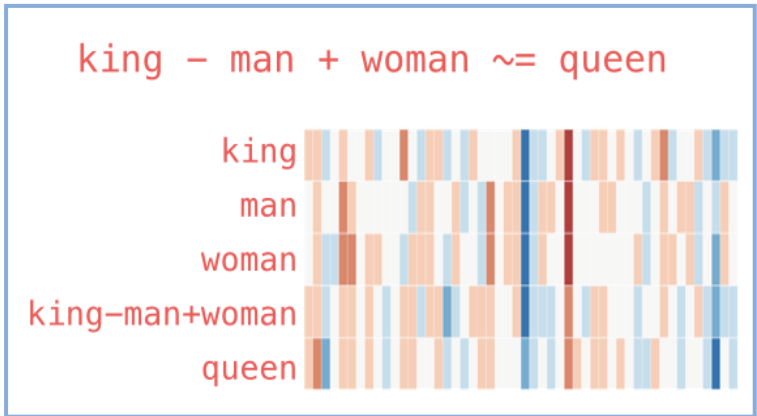
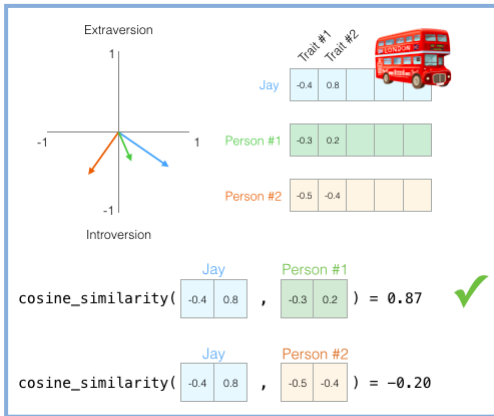


0 (negative) or 1 (positive)



54

QUICK OVERVIEW: Word embedding in NLP



- Other application:

- (16MAR2020) Customer2Vec: Representation learning for customer analytics and personalization
<https://blog.griddynamics.com/customer2vec-representation-learning-and-automl-for-customer-analytics-and-personalization/>

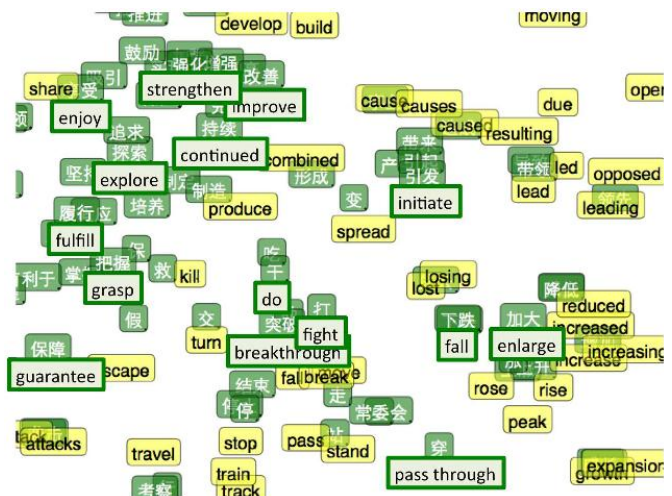
Image credit: <https://jalammar.github.io/illustrated-word2vec/>

55

QUICK OVERVIEW: Word embedding in NLP

- In a generative model where the model predicts a word-embedding vector, how can we map the embedding vector back to its corresponding natural word?

- Since the word-embedding space is continuous, it is possible that the predicted vector cannot be mapped directly with a simple lookup table.
- In Transformer (2017), it shares weights among two embedding layers and one pre-softmax linear layer. The pre-softmax linear layer is defined as
`nn.Linear( in_features=embed_size, out_features=vocab_size )`.
- In huggingface, there is a function like `tokenizer.batch_decode()` that helps interpret a token to a word.



56

AI LSTM for Video

- CNN-LSTM model
- ConvLSTM layer
 - [5JUN2021] Next-Frame Video Prediction with Convolutional LSTMs https://keras.io/examples/vision/conv_lstm/
 - [25MAR2019] An introduction to ConvLSTM <https://medium.com/neuronio/an-introduction-to-convlstm-55c9025563a7>

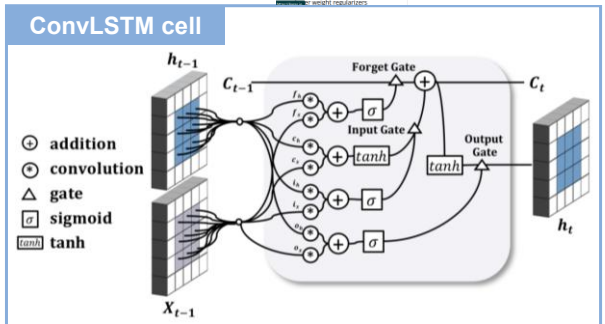
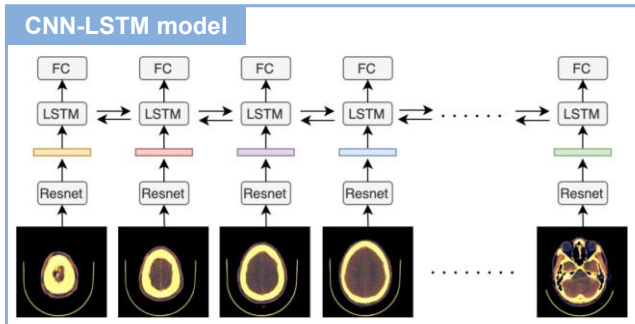


Image credit: <https://www.medrxiv.org/content/10.1101/2020.04.17.20070193v1.full> , <http://homepage.divms.uiowa.edu/~zhuoning/papers/p984-yuan.pdf>

57



Homework

Sequential inputs/outputs

58

Sequence Data: RNN/LSTM/GRU



- **Objective:** What it takes to train and tune RNN/LSTM/GRU models for sequence data.
- Choose one unimodal non-video task:
 - Time series vs. NLP vs. Speech, Generative vs. Discriminative
 - Can be a self-collected dataset or any ready-to-use dataset found on the internet, except for the dataset already mentioned in class examples (**no dataset duplication between groups**)
 - Be sure to use proper evaluation metrics regarding your selected problem
- For any group using a self-collected dataset:
 - Compare results regarding at least three different DL model architectures
 - Provide your dataset in Github, and **be careful not to violate any data privacy rule both domestically and internationally**
- For any group using a ready-to-use dataset found on the internet:
 - Compare results regarding at least three different DL model architectures **and** at least one non-DL technique (e.g., traditional ML, rule-based)



59



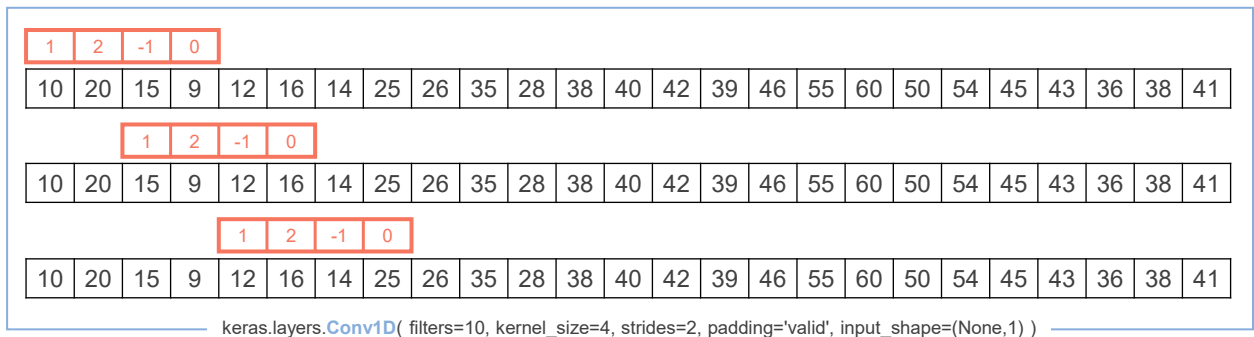
Alternative architectures

Processing sequences without
RNN/LSTM/GRU

60

A Conv1D for sequence processing

- A 1D convolutional layer slides several kernels across a sequence, producing a 1D feature map per kernel.
 - Each kernel will learn to detect a single very short sequential pattern (no longer than the kernel size).
 - If we use ten kernels, then the layer's output will be composed of ten 1D sequences (all of the same length), or equivalently we can view this output as a single 10D sequence.
 - This means that we can build a neural network composed of a mix of recurrent layers and 1D convolutional layers (or even 1D pooling layers).



61

A WaveNet (arxiv 2016)



```

1 model = keras.models.Sequential()
2
3 model.add( keras.layers.InputLayer( input_shape=(None,1) ) )
4 for rate in ( 1, 2, 4, 8 ) * 2 :
5     model.add( keras.layers.Conv1D( filters=20, kernel_size=2, padding='casual',
6                                     activation='relu', dilation_rate=rate )
7     )
8 model.add( keras.layers.Conv1D( filters=10, kernel_size=1 ) )
9
10 model.compile( loss='mse', optimizer='Adam', metrics=[last_time_step_mse] )
11 hist = model.fit( x_train, y_train, epochs=20, validation_data=(x_val, y_val) )

```

Dilated convolution

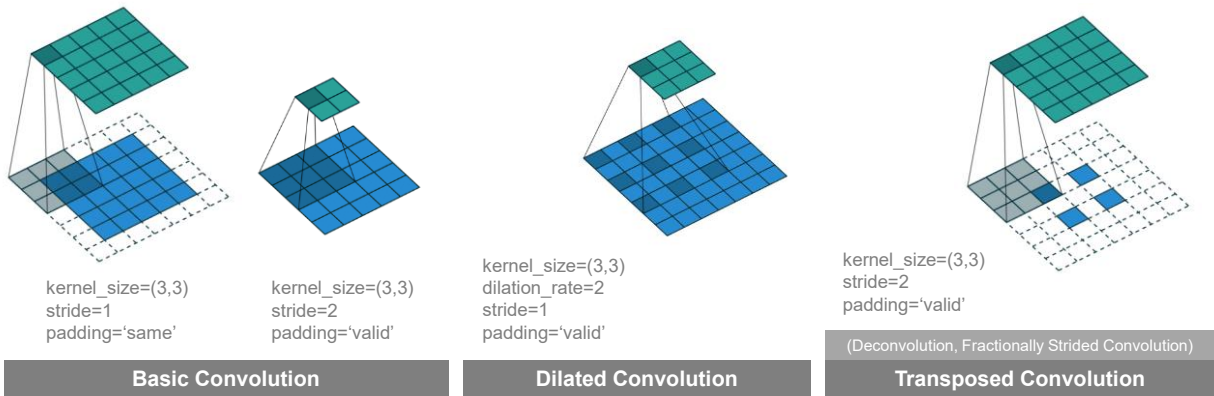
Casual convolution

Google, "WaveNet: A Generative model for raw audio," arXiv, 2016, <https://arxiv.org/abs/1609.03499>

62

Dilated Convolution

- Expands the receptive field w/o increasing the number of parameters by inserting gaps between filter pixels.
- Captures long-range dependencies efficiently while preserving spatial resolution.
- Useful in segmentation and sequence modeling, as it helps recognize patterns over a wider context w/o losing details.

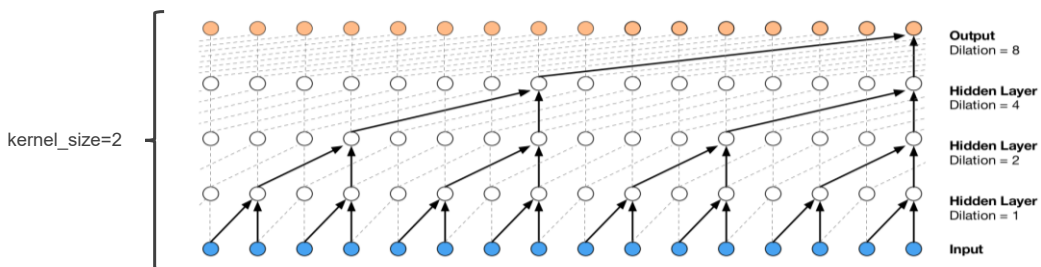


Image(s) credit: <https://towardsdatascience.com/types-of-convolutions-in-deep-learning-717013397f4d>

63

WaveNet (arxiv 2016)

- **WaveNet** stacks 1D convolutional layers, doubling the dilation rate at every layer.
 - The 1st Conv1D layer gets a glimpse of just two time steps at a time.
 - The 2nd Conv1D layer sees four time steps (its receptive field is four time steps long).
 - The 3rd Conv1D layer sees eight time steps.
 - ...so on...
- This means that the lower Conv1D layers learn short-term patterns, while the higher Conv1D layers learn long-term patterns.



Image(s) from: <https://github.com/ShichengChen/Audio-Source-Separation>

64

WaveNet (arxiv 2016)

- In **WaveNet (2016)** paper (by Google's DeepMind):
 - The authors stacked 10 convolutional layers with dilation rates of 1, 2, 4, 8, ..., 256, 512. Then, they stacked another group of 10 identical layers (with the dilation rates 1, 2, 4, 8, ..., 256, 512), then again another identical group of ten layers.
 - The authors pointed out that a single stack of 10 convolutional layers with these dilation rates will act like a super-efficient convolutional layer with a kernel of size 1,024 (except way faster, more powerful, and using significantly fewer parameters).
 - Interesting articles:
 - Audio generation <https://deepmind.com/blog/article/wavenet-generative-model-raw-audio>
 - Google Assistant's voice synthesizer <https://towardsdatascience.com/wavenet-google-assistants-voice-synthesizer-a168e9af13b1>
 - Text-to-Speech <https://medium.com/avangelist/tacotron-2-%E0%B8%8A%E0%B8%B4%E0%B8%94%E0%B9%83%E0%B8%99%E0%B9%80%E0%B8%A5%E0%B8%A2%E0%B9%80%E0%B8%9E%E0%B9%88-9c35fb6a6227>

65

Foundation model for time series

- **TimeGPT**: The first foundation model for time series
 - A cutting-edge generative pre-trained transformer model dedicated to prediction tasks
 - By leveraging the most extensive dataset ever – financial, weather, energy, and sales data – **TimeGPT** brings unparalleled time-series analysis right to your terminal!

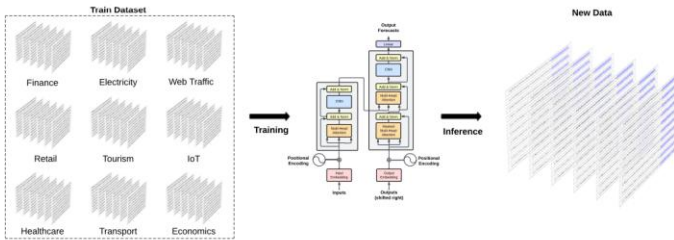


Figure 2: TimeGPT was trained in the largest collection of publicly available time series, and can forecast unseen time series without re-training its parameters.

Table 1: Main performance results for TimeGPT with zero-shot inference and benchmark models measured with rMAE and rRMSE, lower scores are better. The best model for each frequency and metric is highlighted in bold, the second best underlined, and the third best underlined with a dotted line.

	Monthly		Weekly		Daily		Hourly	
	rMAE	rRMSE	rMAE	rRMSE	rMAE	rRMSE	rMAE	rRMSE
ZeroModel	2.045	1.568	6.075	6.075	2.989	2.395	10.255	8.183
HistoricAverage	1.349	1.106	4.188	4.188	2.509	2.057	2.216	1.964
SeasonalNaive	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Theta	0.839	0.764	1.061	1.061	0.841	0.811	1.163	1.175
DOTtheta	0.799	0.734	1.056	1.056	0.837	0.806	1.157	1.169
ETS	0.942	0.960	1.079	1.079	0.944	0.970	0.998	1.009
CES	1.024	0.946	1.002	1.002	0.919	0.899	0.878	0.896
ADIDA	0.852	0.769	1.364	1.364	0.908	0.868	2.307	2.207
IMAPA	0.852	0.769	1.364	1.364	0.908	0.868	2.307	2.207
CrostonClassic	0.989	0.857	1.805	1.805	0.995	0.933	2.157	2.043
LGBM	1.050	0.913	0.993	0.993	2.506	2.054	0.733	0.709
LSTM	0.836	0.778	1.002	1.002	0.852	0.832	0.974	0.955
DeepAR	0.988	0.878	0.987	0.987	0.853	0.826	1.028	1.028
TFT	0.752	0.700	0.954	0.954	0.817	0.791	1.120	1.112
NHITS	0.738	0.694	0.883	0.883	0.788	0.771	0.829	0.860
TimeGPT	0.727	0.685	0.878	0.878	0.804	0.780	0.852	0.878

"TimeGPT-1," arXiv 2023, <https://arxiv.org/abs/2310.03589>, <https://github.com/Nixtla/nixtla>

66

Foundation model for time series

• **TSMamba**: A Mamba Foundation Model for Time Series Forecasting

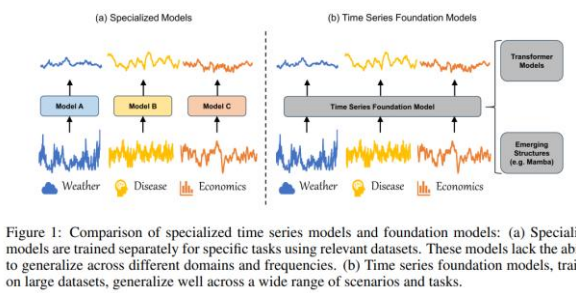


Figure 1: Comparison of specialized time series models and foundation models: (a) Special models are trained separately for specific tasks using relevant datasets. These models lack the ability to generalize across different domains and frequencies. (b) Time series foundation models, trained on large datasets, generalize well across a wide range of scenarios and tasks.

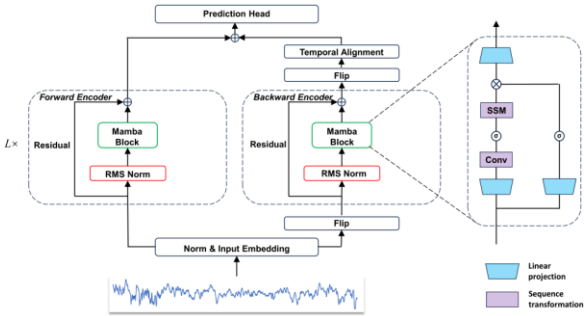


Figure 2: TSMamba Architecture: The input time series are preprocessed and then fed into the forward and backward encoder to extract internal dependencies. The representations are combined and subsequently mapped to forecasts by the prediction head.

5NOV2024: <https://arxiv.org/abs/2411.02941v1>

67



68